Week 1: Data Modeling

AUTHOR
Aakriti Poudel

PUBLISHED
April 7, 2026

Setup

```
# Load all required libraries
library(tidyverse)
library(DBI)
library(duckdb)

# Read data
snow <- read_csv(
  "https://ucsb-library-research-data-services.github.io/bren-eds213/modules/week01/ASDN_
)
```

Exploring the Data

First I look at the data so I can make informed decisions about data types, which columns can be NULL, and what constraints make sense.

Column types and sample values

`glimpse()` shows the column names, how R read each column, and a few sample values. This will help me decide the right SQL data type for each column.

```
glimpse(snow)
```

```
Rows: 27,936
Columns: 11
$ Site        <chr> "barr", "barr", "barr", "barr", "barr", "barr", "barr", "b…
$ Year        <dbl> 2011, 2011, 2011, 2011, 2011, 2011, 2011, 2011, 2011, 2011…
$ Date        <date> 2011-05-29, 2011-05-29, 2011-05-29, 2011-05-29, 2011-05-2…
$ Plot        <chr> "brw1", "brw1", "brw1", "brw1", "brw1", "brw1", "brw1", "b…
$ Location    <chr> "b10", "b12", "b2", "b4", "b6", "b8", "d10", "d12", "d2", …
$ Snow_cover  <dbl> 90, 100, 90, 100, 95, 95, 95, 90, 95, 95, 90, 90, 95, 90, …
$ Water_cover <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10, 0, 0, 0, 0, 0, 0, 0, 0, …
$ Land_cover  <dbl> 10, 0, 10, 0, 5, 5, 5, 10, 5, 5, 0, 10, 5, 10, 20, 10, 5, …
$ Total_cover <dbl> 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100…
$ Observer    <chr> "adoll", "adoll", "adoll", "adoll", "adoll", "adoll", "ado…
$ Notes       <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA…
```

Let's print a few unique values from each column to see what the data actually looks like.

```r
peek <- function(x, label, n = 5) {
  cat(label, ":", paste(head(unique(x), n), collapse = ", "), "\n")
}

peek(snow$Site, "Site")
```

```
Site : barr, cakr, cari, chur, colv
```

```r
peek(snow$Year, "Year")
```

```
Year : 2011, 2012, 2013, 2014, 2010
```

```r
peek(snow$Date, "Date")
```

```
Date : 2011-05-29, 2011-05-31, 2011-06-02, 2011-06-03, 2011-06-04
```

```r
peek(snow$Plot, "Plot")
```

```
Plot : brw1, brw2, brw6, brw3, brw5
```

```r
peek(snow$Location, "Location")
```

```
Location : b10, b12, b2, b4, b6
```

```r
peek(snow$Snow_cover, "Snow_cover")
```

```
Snow_cover : 90, 100, 95, 80, 70
```

```r
peek(snow$Water_cover, "Water_cover")
```

```
Water_cover : 0, 10, 5, 15, 25
```

```r
peek(snow$Land_cover, "Land_cover")
```

```
Land_cover : 10, 0, 5, 20, 15
```

```r
peek(snow$Total_cover, "Total_cover")
```

```
Total_cover : 100
```

```r
peek(snow$Observer, "Observer")
```

```
Observer : adoll, jcunningham, abankert, bhill, pherzog
```

```
peek(snow$Notes, "Notes")
```

```
Notes : NA, water estimate suspect, data collection error, Probably an observer error.
Maybe should have been 100 snow., water cover estimates suspect, may be swapped with
land.
```

From these, I can see that `Site` is a short text code, cover columns have decimal values, and `Date` is a real date, not just a number.

Missing values (NULL check)

Here, check the count of missing values in each column.

```
colSums(is.na(snow))
```

```
       Site        Year        Date        Plot    Location  Snow_cover
          0           0           0         154           9           0
Water_cover  Land_cover Total_cover    Observer       Notes
          0           0           0           0       27128
```

Only `Location`, `Observer`, and `Notes` have missing values. Everything else has zero, so those columns should be `NOT NULL`.

Primary key

A primary key must uniquely identify every row. I test whether the combination of `Site`, `Date`, `Plot`, and `Location` works. If this returns no rows, it means no duplicates exist and this combination is a valid primary key.

```
snow |>
  count(Site, Date, Plot, Location) |>
  filter(n > 1)
```

```
# A tibble: 207 × 5
   Site  Date       Plot  Location      n
   <chr> <date>     <chr> <chr>     <int>
 1 cakr  2013-06-19 nepe  nepe rsc4     2
 2 eaba  2013-06-15 <NA>  sno01         2
 3 eaba  2013-06-15 <NA>  sno03         2
 4 eaba  2013-06-15 <NA>  sno04         2
 5 eaba  2013-06-15 <NA>  sno09         2
 6 eaba  2013-06-15 <NA>  sno10         2
 7 made  2011-06-06 taglu e8            2
 8 prba  2012-06-14 9     1             2
 9 prba  2012-06-14 9     10            2
10 prba  2012-06-14 9     11            2
# ℹ 197 more rows
```

No duplicates. Now, check what happens if I drop `Location` from the key, if duplicates appear, then `Location` is necessary.

```
snow |>
  count(Site, Date, Plot) |>
  filter(n > 1) |>
  head()
```

```
# A tibble: 6 × 4
  Site  Date       Plot      n
  <chr> <date>     <chr> <int>
1 barr  2006-06-04 brw1     36
2 barr  2006-06-04 brw2     16
3 barr  2006-06-04 brw3     36
4 barr  2006-06-04 brw5     36
5 barr  2006-06-04 brw6     12
6 barr  2007-06-05 brw3     13
```

Duplicates appear without `Location`, so it must stay in the key.

Cover value ranges

The instructions ask about value constraints. Cover columns represent percentages, so check whether any value falls outside the 0 to 100 range. If this returns no rows, the constraint is safe to add.

```
snow |>
  filter(Snow_cover < 0   | Snow_cover > 100 |
         Water_cover < 0  | Water_cover > 100 |
         Land_cover < 0   | Land_cover > 100)
```

```
# A tibble: 0 × 11
# ℹ 11 variables: Site <chr>, Year <dbl>, Date <date>, Plot <chr>,
#   Location <chr>, Snow_cover <dbl>, Water_cover <dbl>, Land_cover <dbl>,
#   Total_cover <dbl>, Observer <chr>, Notes <chr>
```

No values outside the range, so `BETWEEN 0 AND 100` is a valid constraint for all three cover columns.

Cover columns sum check

The metadata says `Total_cover` is a checksum, it should always equal the sum of the three cover parts. Verify if this is true in the data.

```
snow |>
  mutate(total_check = Snow_cover + Water_cover + Land_cover) |>
  filter(abs(total_check - Total_cover) > 0.01)
```

```
# A tibble: 0 × 12
# ℹ 12 variables: Site <chr>, Year <dbl>, Date <date>, Plot <chr>,
#   Location <chr>, Snow_cover <dbl>, Water_cover <dbl>, Land_cover <dbl>,
#   Total_cover <dbl>, Observer <chr>, Notes <chr>, total_check <dbl>
```

```
unique(snow$Total_cover)
```

```
[1] 100
```

The three parts always add up to `Total_cover`, and `Total_cover` is always 100 in every row. Both constraints are safe to enforce.

Year and date consistency

`Year` and `Date` both describe when the survey happened. They should always agree. Check for any rows where `Year` does not match the year part of `Date`.

```
snow |>
  mutate(Date = as.Date(Date)) |>
  filter(Year != year(Date))
```

```
# A tibble: 0 × 11
# ℹ 11 variables: Site <chr>, Year <dbl>, Date <date>, Plot <chr>,
#   Location <chr>, Snow_cover <dbl>, Water_cover <dbl>, Land_cover <dbl>,
#   Total_cover <dbl>, Observer <chr>, Notes <chr>
```

No mismatches. We can safely add a constraint that requires `Year = year(Date)`.

SQL Table Definition

Based on everything above, here is the `Snow_survey` table definition using DuckDB syntax. Each constraint reflects a specific finding from the exploration.

```
con <- dbConnect(duckdb())

dbExecute(con, "
CREATE TABLE Snow_survey (

    Site        TEXT    NOT NULL,
    Year        INTEGER NOT NULL,
    Date        DATE    NOT NULL,
    Plot        TEXT    NOT NULL,
    Location    TEXT,
    Snow_cover  REAL    NOT NULL,
    Water_cover REAL    NOT NULL,
    Land_cover  REAL    NOT NULL,
    Total_cover REAL    NOT NULL,
    Observer    TEXT,
    Notes       TEXT,

    PRIMARY KEY (Site, Date, Plot, Location),

    -- skipping FK for now, Site and Observer reference other tables,
```

```
    -- FOREIGN KEY (Site)     REFERENCES Site(Code),
    -- FOREIGN KEY (Observer) REFERENCES Personnel(Abbreviation),

    CONSTRAINT year_range
        CHECK (Year BETWEEN 1900 AND 2100),

    CONSTRAINT year_matches_date
        CHECK (Year = year(Date)),

    CONSTRAINT snow_cover_range
        CHECK (Snow_cover  BETWEEN 0 AND 100),

    CONSTRAINT water_cover_range
        CHECK (Water_cover BETWEEN 0 AND 100),

    CONSTRAINT land_cover_range
        CHECK (Land_cover  BETWEEN 0 AND 100),

    CONSTRAINT total_cover_equals_100
        CHECK (Total_cover = 100),

    CONSTRAINT cover_parts_sum_to_total
        CHECK (ABS((Snow_cover + Water_cover + Land_cover) - Total_cover) < 0.01)
);
")
```

```
[1] 0
```

`DESCRIBE` shows the final schema: column names, types and whether NULL is allowed.

```
dbGetQuery(con, "DESCRIBE Snow_survey")
```

```
   column_name column_type null  key default extra
1         Site     VARCHAR   NO  PRI    <NA>  <NA>
2         Year     INTEGER   NO <NA>    <NA>  <NA>
3         Date        DATE   NO  PRI    <NA>  <NA>
4         Plot     VARCHAR   NO  PRI    <NA>  <NA>
5     Location     VARCHAR   NO  PRI    <NA>  <NA>
6   Snow_cover       FLOAT   NO <NA>    <NA>  <NA>
7  Water_cover       FLOAT   NO <NA>    <NA>  <NA>
8   Land_cover       FLOAT   NO <NA>    <NA>  <NA>
9  Total_cover       FLOAT   NO <NA>    <NA>  <NA>
10    Observer     VARCHAR  YES <NA>    <NA>  <NA>
11       Notes     VARCHAR  YES <NA>    <NA>  <NA>
```

Load the actual data into the table to confirm everything works.

```
snow_clean <- snow |>
  select(Site, Year, Date, Plot, Location,
         Snow_cover, Water_cover, Land_cover, Total_cover,
```

```
        Observer, Notes)

dbWriteTable(con, "Snow_survey", snow_clean, overwrite = TRUE)

dbGetQuery(con, "SELECT COUNT(*) AS total_rows FROM Snow_survey")
```

```
  total_rows
1      27936
```

```
dbGetQuery(con, "SELECT * FROM Snow_survey LIMIT 10")
```

```
   Site Year       Date Plot Location Snow_cover Water_cover Land_cover
1  barr 2011 2011-05-29 brw1      b10         90           0         10
2  barr 2011 2011-05-29 brw1      b12        100           0          0
3  barr 2011 2011-05-29 brw1       b2         90           0         10
4  barr 2011 2011-05-29 brw1       b4        100           0          0
5  barr 2011 2011-05-29 brw1       b6         95           0          5
6  barr 2011 2011-05-29 brw1       b8         95           0          5
7  barr 2011 2011-05-29 brw1      d10         95           0          5
8  barr 2011 2011-05-29 brw1      d12         90           0         10
9  barr 2011 2011-05-29 brw1       d2         95           0          5
10 barr 2011 2011-05-29 brw1       d4         95           0          5
   Total_cover Observer Notes
1          100    adoll  <NA>
2          100    adoll  <NA>
3          100    adoll  <NA>
4          100    adoll  <NA>
5          100    adoll  <NA>
6          100    adoll  <NA>
7          100    adoll  <NA>
8          100    adoll  <NA>
9          100    adoll  <NA>
10         100    adoll  <NA>
```

Explanation

Data types

- `Site`, `Plot`, `Location`, `Observer`, and `Notes` are `TEXT` because they are names or short codes, not numbers you would do math with.
- `Year` is `INTEGER` because it is a whole number and you might want to filter or sort by it.
- `Date` is `DATE` so the database treats it as a real date. This lets us use date functions like `year()`. Storing it as `TEXT` would make that impossible.
- The four cover columns are `REAL` because the values have decimals (e.g., 95.0, 7.5).

Primary key

No single column on its own identifies a row uniquely. A survey row is defined by where it happened (site + plot + location) and when (date). The R checks above confirmed that `(Site, Date, Plot, Location)` has zero duplicates, and that removing `Location` creates duplicates. So the primary key is composite and `Location` must be part of it.

Foreign keys

`Site` looks like it references the `Code` column in the `Site` table, and `Observer` looks like it references `Abbreviation` in the `Personnel` table. These relationships exist in the full ASDN dataset. I left them as comments in the SQL because we are only creating the `Snow_survey` table here.

NULL values

I checked with `colSums(is.na(snow))`. Only `Location`, `Observer`, and `Notes` have missing values in the data, so only those three are allowed to be NULL. All other columns are `NOT NULL`.

Uniqueness

No single column has unique values across all rows, so there are no single-column `UNIQUE` constraints. The composite primary key is the only uniqueness rule needed.

Other constraints

- Each cover column is constrained to `BETWEEN 0 AND 100` because they are percentages.
- `Total_cover` is constrained to equal 100. The metadata says this is a checksum column and every row in the data confirms this.
- Snow, water, and land cover must add up to `Total_cover`. I use a small tolerance of 0.01 to avoid false failures from floating point rounding.
- `Year` must equal `year(Date)` because both columns describe the same date and should never contradict each other.
- Each constraint has a name so that if a row is rejected, the error message says exactly which rule was broken.

Questions and uncertainties

- Could the same plot be surveyed twice on the same day by two different observers? If yes, the primary key I chose would not work and `Observer` would need to be added to it.
- All cover values in the data are multiples of 5 (0, 5, 10, …), which suggests the field protocol uses 5% intervals. I chose not to enforce this because a future observer might record a finer value and we should not reject that.